Week 12: Cryptography Lab Activities & Practical Assignments

CCS6324 - Ethical Hacking & Penetration Testing

Trimester 2530

Total Assessment Weight: 4% of final grade

Lab Overview

This week focuses on practical cryptography skills: password cracking, hash analysis, certificate management, encryption/decryption, and cryptographic weakness identification. All activities are designed for students with minimal prior experience but require careful attention to detail and ethical considerations.

Lab Ethical Guidelines

- All activities are for **educational purposes only**
- Perform password cracking **only on systems you own or have explicit written permission**
- Never test on production systems or other users' accounts
- Unauthorized password cracking is illegal in most jurisdictions
- Report security findings responsibly (responsible disclosure)

Lab Activity 1: Password Cracking with John the Ripper

Difficulty: Beginner | Time: 30 minutes

Objectives

- Install and configure John the Ripper
- Understand password hash formats
- Execute dictionary and brute force attacks
- Analyze password strength

Prerequisites

- Linux/Windows system
- Basic command-line familiarity
- Administrative/sudo access

Step-by-Step Instructions

Part A: Installation & Setup

Windows Users:

```
1. Visit https://www.openwall.com/john/
2. Download "John the Ripper Community" (Windows builds)
3. Extract to C:\john-1.9.0-jumbo-1
4. Open Command Prompt as Administrator
5. Navigate: cd C:\john-1.9.0-jumbo-1\run
6. Verify installation: john.exe --version
```

Linux Users:

```
1. Open terminal
2. Install via package manager:
   sudo apt-get update
   sudo apt-get install john john-data
3. Verify: john --version
4. For GPU acceleration (optional):
   sudo apt-get install john-cuda
```

Part B: Creating Test Password Hashes

Step 1: Generate Sample Hashes

On Linux, create test hashes:

```
# Using openssl to create MD5 hash
echo -n "password123" | md5sum
# Output: 482c811da5d5b4bc6d497ffa98491e38

# Using openssl for SHA-256
echo -n "password123" | sha256sum
# Output: ef92b778bafe771e89245d171bafed20891fe8147519c94cec084b8e61342d659
```

Step 2: Create Hash File

Create file `test_hashes.txt` with sample hashes:

```
# MD5 hashes (format: username:hash)
john:482c811da5d5b4bc6d497ffa98491e38
admin:209c6174da490caeb422f3fa5a7ae634
```

```
user1:5f4dcc3b5aa765d61d8327deb882cf99
root:5d41402abc4b2a76b9719d911017c592

# Comments: Passwords are: password123, password, password, hello
```

Part C: Dictionary Attack

Step 1: Obtain Wordlist

```
# On Linux (built-in):
ls /usr/share/wordlists/

# If not found, download:
sudo apt-get install wordlist
# or
wget https://github.com/danielmiessler/SecLists/raw/master/Passwords/Common-
Credentials/10k-most-common.txt

# On Windows:
# Download from above URL, save to john directory
```

Step 2: Run Dictionary Attack

```
# Linux/macOS/Windows (PowerShell):
john --wordlist=/usr/share/wordlists/rockyou.txt test_hashes.txt

# Or using built-in rockyou.txt from John:
john --wordlist=password.lst test_hashes.txt

# Expected output:
# Loaded 4 password hashes with no differently salted hashes
# password (user1)
# hello (root)
# password123 (john)
# password (admin)
# 4 password hashes cracked, 0 left
```

Step 3: View Results

```
# Show cracked passwords:
john --show test_hashes.txt

# Output format:
# john:password123
# admin:password
# user1:password
# root:hello
```

Part D: Brute Force Attack

Step 1: Understand Keyspace

Create limited-scope hash:

```
# Create hash with weak password we'll brute force
echo -n "abc123" | md5sum
# Output: 0d7ce4fdf2ae7ca51686c7e105f0b4ac
```

Step 2: Configure Brute Force

Create `john.conf` section:

```
# Create working configuration
john --incremental=Digits --min-length=3 --max-length=6 \
     --format=Raw-MD5 test_hashes.txt
```

Step 3: Run Attack

```
# Brute force numeric passwords 3-6 characters
john --incremental=Digits test_single_hash.txt

# Monitor progress:
# It will attempt: 000, 001, 002, ... abc, abd, ... zzz...
# Once found:
# abc123 (0d7ce4fdf2ae7ca51686c7e105f0b4ac)
```

Step 4: Analyze Results

```
# Check session status while running:
john --status

# Resume interrupted session:
john --restore

# Output statistics:
john --show --verbose test_hashes.txt
```

Part E: Performance Analysis

Document the Results:

Create report covering:

1. **Dictionary Attack:**

- Passwords cracked: _
- Time taken: _
- Success rate: _%
- Computational cost: _

1. **Brute Force Attack:**

- Passwords cracked: _
- Time taken: _

- Attempts per second: _
- Key space covered: _%

1. **Observations:**

- Which attack was faster? Why?
- What determines attack speed? (wordlist size, key space, hardware)
- How would attack change with salted hashes?

Deliverables

- Cracked passwords with hashes
- Execution screenshots
- Performance analysis document
- Summary of key findings

Lab Activity 2: Hash Analysis & Collision Detection

Difficulty: Beginner-Intermediate | Time: 45 minutes

Objectives

- Compare hash function outputs
- Understand avalanche effect
- Identify weak hashing algorithms
- Detect collision vulnerabilities

Prerequisites

- Linux/macOS or Windows with OpenSSL installed
- Text editor
- Basic hex understanding

Step-by-Step Instructions

Part A: Install Hashing Tools

Windows:

```
# Install OpenSSL (if not present)
1. Visit http://slproweb.com/products/Win32OpenSSL.html
2. Download "Win64 OpenSSL Light"
3. Install to C:\Program Files\OpenSSL-Win64
4. Add to PATH: C:\Program Files\OpenSSL-Win64\bin
```

```
5. Open Command Prompt: openssl version
```

Linux/macOS:

```
# Already installed; verify:
openssl version

# If needed:
sudo apt-get install openssl  # Ubuntu/Debian
brew install openssl           # macOS
```

Part B: Generate Hashes with Different Algorithms

Step 1: Create Test File

```
# Create test document
echo "The quick brown fox jumps over the lazy dog" > document.txt

# Create similar document (1 character difference)
echo "The quick brown fox jumps over the lazy cog" > document_modified.txt
```

Step 2: Generate MD5 Hashes

```
# Linux/macOS:
md5 document.txt
md5 document_modified.txt

# Windows (PowerSSL):
openssl dgst -md5 document.txt
openssl dgst -md5 document_modified.txt

# Expected Output:
# MD5(document.txt)= d41d8cd98f00b204e9800998ecf8427e
# MD5(document_modified.txt)= 6d7fce9fee471194aa8b5b6ff7d92cf8
```

Step 3: Generate SHA-1 Hashes

```
# Linux/macOS:
shasum document.txt
shasum document_modified.txt

# Windows:
openssl dgst -sha1 document.txt
openssl dgst -sha1 document_modified.txt

# Note: Output varies significantly from original
```

Step 4: Generate SHA-256 Hashes

```
# Linux/macOS:
shasum -a 256 document.txt
shasum -a 256 document_modified.txt
```

```
# Windows:
openssl dgst -sha256 document.txt
openssl dgst -sha256 document_modified.txt
```

Part C: Analyze Avalanche Effect

Create Analysis Table:

```
Document: "The quick brown fox jumps over the lazy dog"
Modified Document: "The quick brown fox jumps over the lazy cog"
(Changed character: 'd' → 'c' - only 1 bit different)

Algorithm | Original Hash (first 32 chars) | Modified Hash (first 32 chars) |
Bits Changed
-----------|------|------|------
MD5        | d41d8cd98f00b204e9800998ecf | 6d7fce9fee471194aa8b5b6ff7d92 |

___
SHA-1      | 2fd4e1c67a2d28fced849ee1bb76e7be |
1fd4e1c67a2d28fced849ee1bb76e7be | ___
SHA-256    | d7a8fbb3... | a3ffc3ba... | ___
```

Calculation Steps:

1. Write both hashes in binary
2. Compare bit-by-bit
3. Count differing bits
4. Record percentage changed

Expected Results:

- Good hash: ~50% bits differ (true avalanche effect)
- Any algorithm: Total hash output changes dramatically

Part D: Collision Research

Part D1: Historical MD5 Collision

Research and document:

```
Known MD5 Collision (2008):
File 1: [specific binary content]
File 2: [specific binary content]
Both produce same MD5: ?

Your task:
1. Find online documented MD5 collision
2. Verify it using: openssl dgst -md5 file1 file2
3. Document: Why is this a security vulnerability?
```

Part D2: Generate Vulnerable Hash Scenario

Create vulnerable scenario:

```
# Example code for hash collision demonstration
# (Conceptual - actual collision generation is complex)

# Attempt with different input encodings:
echo -n "admin" | md5sum
echo "admin" | md5sum
printf "admin" | md5sum

# All produce same hash or different hashes?
# Why might this matter in real applications?
```

Part E: Hash Algorithm Comparison

Complete Comparison Table:

Property	MD5	SHA-1	SHA-256	SHA-512
Output Size	_	_	_	_
Status	✗	✗	☑	☑
Time to Generate	_	_	_	_
Collision Risk	HIGH	HIGH	LOW	LOW
Avalanche Effect	GOOD	GOOD	GOOD	GOOD

Performance Benchmarking:

```
# Time hash generation for large file
time openssl dgst -md5 largefile.bin
time openssl dgst -sha256 largefile.bin
time openssl dgst -sha512 largefile.bin

# Record results and analyze speed vs. security trade-off
```

Deliverables

- Hash comparison table with calculations
- Avalanche effect analysis with percentages
- Historical collision documentation
- Performance benchmark results
- Recommendations for hash algorithm selection

Lab Activity 3: OpenSSL Encryption & Decryption

Difficulty: Intermediate | Time: 60 minutes

Objectives

- Use OpenSSL for symmetric encryption
- Apply asymmetric encryption
- Manage digital certificates
- Understand encryption modes

Prerequisites

- OpenSSL installed and functional
- Basic understanding of public/private keys
- File system access

Step-by-Step Instructions

Part A: Symmetric Encryption with AES

Step 1: Create Test Document

```
# Create sensitive document
cat > secret_message.txt << EOF
CONFIDENTIAL
Project Code: APOLLO
Launch Date: 2024-03-15
Budget: $2.5M
Contact: alice@company.com
EOF
```

Step 2: Encrypt with AES-256

```
# Encryption with password
openssl enc -aes-256-cbc -in secret_message.txt \
  -out secret_message.txt.enc -P

# Terminal prompts:
# Enter password: [user enters password]
# Verifying password: [user re-enters]
# Output shows:
# salt=XXXXXXXX...
# key=XXXXXXXX...
# iv=XXXXXXXX...
```

Step 3: Examine Encrypted File

```
# View encrypted file (will be binary/unreadable)
```

```
cat secret_message.txt.enc

# Check file properties
ls -la secret_message.txt*

# Note: Encrypted file is much larger (includes salt/IV)
```

Step 4: Decrypt File

```
# Decrypt (prompts for password)
openssl enc -aes-256-cbc -d -in secret_message.txt.enc \
  -out secret_message_decrypted.txt

# Verify decryption worked
cat secret_message_decrypted.txt

# Compare with original
diff secret_message.txt secret_message_decrypted.txt
# Output: (no differences = successful encryption/decryption)
```

Step 5: Explore Different Cipher Modes

```
# ECB mode (INSECURE - demonstrating why):
openssl enc -aes-256-ecb -in plaintext.txt -out ecb_encrypted.bin

# CBC mode (STANDARD SECURE):
openssl enc -aes-256-cbc -in plaintext.txt -out cbc_encrypted.bin

# GCM mode (AUTHENTICATED ENCRYPTION - most secure):
openssl enc -aes-256-gcm -in plaintext.txt -out gcm_encrypted.bin

# Compare file sizes and document differences
```

Part B: Asymmetric Encryption with RSA

Step 1: Generate RSA Key Pair

```
# Generate 2048-bit private key
openssl genrsa -out alice_private.key 2048

# Output: Generating RSA private key...
# 2048 bit long modulus

# Extract public key from private key
openssl rsa -in alice_private.key -pubout -out alice_public.key

# Display keys (truncated):
cat alice_public.key
# -----BEGIN PUBLIC KEY-----
# MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A...
# -----END PUBLIC KEY-----
```

Step 2: Encrypt with Public Key

```
# Create message
echo "Meet at location X tonight" > message.txt

# Encrypt using Alice's public key
openssl pkeyutl -encrypt -in message.txt \
  -pubin -inkey alice_public.key -out message.enc

# Verify encryption
cat message.enc  # Binary output - unreadable
ls -la message*  # Encrypted file is larger
```

Step 3: Decrypt with Private Key

```
# Only Alice can decrypt with her private key
openssl pkeyutl -decrypt -in message.enc \
  -inkey alice_private.key -out message_decrypted.txt

# Verify successful decryption
cat message_decrypted.txt

# Confirm matches original
diff message.txt message_decrypted.txt
```

Step 4: Demonstrate Key Pair Dependency

```
# Attempt to decrypt with wrong key (Bob's private key)
openssl rsa -in alice_private.key -pubout | \
  openssl pkeyutl -decrypt -in message.enc -inkey bob_private.key

# Result: Error - decryption fails with wrong key
# This proves: only correct key pair can decrypt
```

Part C: Digital Signatures

Step 1: Create Document

```
# Create financial document
cat > financial_report.txt << EOF
Q4 2023 Financial Summary
Revenue: $5.2M
Expenses: $3.8M
Net Profit: $1.4M
Prepared by: Alice Chen
Date: 2024-01-15
EOF
```

Step 2: Sign Document

```
# Create signature using Alice's private key
openssl dgst -sha256 -sign alice_private.key \
  financial_report.txt > financial_report.sig

# Verify signature file was created
```

```
ls -la financial_report.sig
```

Step 3: Verify Signature

```
# Verify using Alice's public key
openssl dgst -sha256 -verify alice_public.key \
  -signature financial_report.sig financial_report.txt

# Output: Verified OK
# If document modified, would output: Verification Failure
```

Step 4: Demonstrate Tamper Detection

```
# Modify document
sed -i 's/1.4M/10M/' financial_report.txt

# Attempt verification with modified document
openssl dgst -sha256 -verify alice_public.key \
  -signature financial_report.sig financial_report.txt

# Output: Verification Failure
# Shows: Any tampering detected; signature becomes invalid
```

Deliverables

- Encrypted/decrypted file pairs
- RSA key pair files
- Signed and verified documents
- Detailed encryption/decryption process documentation
- Analysis of cipher modes and their security implications

Lab Activity 4: SSL/TLS Certificate Analysis

Difficulty: Intermediate | Time: 75 minutes

Objectives

- Generate self-signed certificates
- Analyze certificate contents
- Understand certificate chains
- Identify certificate vulnerabilities

Prerequisites

- OpenSSL installed
- Web browser (Chrome/Firefox)

- Text editor

Step-by-Step Instructions

Part A: Generate Self-Signed Certificate

Step 1: Create Private Key

```
# Generate 2048-bit private key for web server
openssl genrsa -out server_private.key 2048

# Output: Generating RSA private key, 2048 bit long modulus...
```

Step 2: Create Certificate Signing Request (CSR)

```
# CSR is submitted to Certificate Authority for signing
openssl req -new -key server_private.key -out server.csr

# Interactive prompts:
# Country Name: US
# State or Province: California
# Locality: San Francisco
# Organization: Example Corp
# Organizational Unit: IT Security
# Common Name: www.example.com
# Email: admin@example.com
# Challenge password: [leave blank]
# Optional company name: [leave blank]
```

Step 3: Create Self-Signed Certificate

```
# Self-sign for testing (normally CA would sign)
openssl x509 -req -days 365 -in server.csr \
  -signkey server_private.key -out server_cert.pem

# Output: Certificate request self-signed successfully
# Valid for 365 days
```

Step 4: Verify Certificate

```
# Display certificate details
openssl x509 -in server_cert.pem -text -noout

# Output shows:
# Certificate:
#   Data:
#     Version: 1 (0x0)
#     Serial Number: ...
#     Signature Algorithm: sha256WithRSAEncryption
#     Issuer: C=US, ST=California...
#     Validity: Not Before: Jan 1 2024, Not After: Jan 1 2025
#     Subject: C=US, ST=California...
#     Public-Key: (2048 bit)
```

```
#     Signature Algorithm: sha256WithRSAEncryption
```

Part B: Analyze Certificate Chain

Step 1: Retrieve Real-World Certificate

```
# Download Google's SSL certificate
openssl s_client -connect google.com:443 < /dev/null 2>/dev/null | \
  openssl x509 -out google_cert.pem

# Or download multiple certificates (full chain)
openssl s_client -connect google.com:443 -showcerts < /dev/null 2>/dev/null |
\
  sed -ne '/-BEGIN CERTIFICATE-/,/-END CERTIFICATE-/p' > chain.txt
```

Step 2: Extract Certificate Information

```
# View Google's certificate details
openssl x509 -in google_cert.pem -text -noout

# Key information to document:
# - Issuer (which CA signed it)
# - Subject (domain, organization)
# - Validity period
# - Public key size
# - Signature algorithm
# - Serial number
# - Extensions (alternative names, constraints)
```

Step 3: Check Certificate Expiration

```
# Get expiration date
openssl x509 -in google_cert.pem -enddate -noout

# Output: notAfter=Jan 15 2025 GMT

# Calculate days until expiration
openssl x509 -in google_cert.pem -noout -dates

# Output:
# notBefore=Jan 10 2024 GMT
# notAfter=Jan 15 2025 GMT
```

Step 4: Analyze Certificate Chain

```
# View complete chain from server to root
openssl s_client -connect google.com:443 -showcerts

# Documents (from deepest to shallowest):
# 1. Leaf certificate (domain certificate)
# 2. Intermediate CA certificate
# 3. Root CA certificate (self-signed, trusted by browsers)
```

```
# Each signs the one below it in the chain
```

Part C: Identify Certificate Vulnerabilities

Step 1: Check for Weak Signature Algorithms

```
# Look for deprecated algorithms in certificate
openssl x509 -in test_cert.pem -text | grep -i "signature"

# VULNERABLE (if present):
# - md5WithRSAEncryption
# - sha1WithRSAEncryption

# SECURE:
# - sha256WithRSAEncryption
# - sha512WithRSAEncryption
```

Step 2: Verify Key Size

```
# Extract key size
openssl x509 -in test_cert.pem -noout -text | grep "Public-Key"

# MINIMUM SECURE:
# - RSA: 2048 bits
# - ECC: 256 bits

# VULNERABLE:
# - RSA: < 2048 bits
# - Any export-grade keys (512-bits or less)
```

Step 3: Check Subject Alternative Names (SANs)

```
# Verify certificate covers intended domains
openssl x509 -in test_cert.pem -text | grep -A1 "Subject Alternative Name"

# Output:
# DNS:www.example.com, DNS:example.com, DNS:mail.example.com

# Verify covered domains match actual usage
```

Step 4: Test Certificate in Browser

Windows/macOS:

```
1. Double-click server_cert.pem
2. Review certificate details window
3. Check: Issuer, validity period, warnings
4. Note: Self-signed = browser warning (expected for testing)
```

Linux:

```
# View in OpenSSL's text form
```

```
openssl x509 -in server_cert.pem -text -noout | less
```

Part D: Certificate Vulnerability Report

Create Comprehensive Report Covering:

```
## Certificate Analysis Report

### Certificate Information
- Subject: _____________
- Issuer: _____________
- Validity: From __________ to __________
- Serial Number: _____________
- Public Key Type: _____________
- Public Key Size: _____________ bits

### Security Assessment
1. Signature Algorithm: _____________
   - Status: ☑ Secure / ⚠️ Weak / ❌ Broken
   - Recommendation: _____________

2. Key Size: _____________ bits
   - Status: ☑ Secure / ⚠️ Adequate / ❌ Weak
   - Recommendation: _____________

3. Validity Period: _____________ days
   - Status: ☑ Appropriate / ⚠️ Too Long / ❌ Expired
   - Recommendation: _____________

4. Issuer Type: _____________
   - Self-signed: Yes/No
   - Trusted CA: Yes/No

### Vulnerabilities Found
[List any identified weaknesses]

### Recommendations
[List improvements needed]
```

Deliverables

- Generated certificate files (.key, .csr, .pem)
- Certificate analysis reports
- Chain verification documentation
- Vulnerability assessment with recommendations
- Screenshots of certificate details

Lab Activity 5: PGP/GPG Key Management

Difficulty: Intermediate | Time: 60 minutes

Objectives

- Generate GPG key pairs
- Encrypt/decrypt files
- Sign and verify messages
- Manage key trust

Prerequisites

- GPG installed (GNU Privacy Guard)
- Text editor
- Understanding of public/private keys

Step-by-Step Instructions

Part A: Install GPG

Windows:

```
1. Visit https://www.gpg4win.org/
2. Download Gpg4win (includes Kleopatra GUI)
3. Install to C:\Program Files (x86)\GnuPG
4. Verify: gpg --version
```

Linux:

```
sudo apt-get update
sudo apt-get install gnupg gnupg2
gpg --version
```

macOS:

```
brew install gnupg
gpg --version
```

Part B: Generate GPG Key Pair

Step 1: Create Master Key

```
# Start key generation
gpg --gen-key

# Or use newer interface
gpg --full-generate-key

# Prompts:
# Key type: RSA and RSA (option 1)
```

```
# Key size: 4096 bits (maximum security)
# Validity: 2y (2 years standard)
# Real name: Alice Johnson
# Email: alice@example.com
# Comment: My Work Key
# Passphrase: [strong passphrase - write down]
# Passphrase again: [confirm]

# Output:
# gpg (GnuPG) 2.x.x
# Generating a 4096-bit RSA key pair...
# [Takes 1-2 minutes on modern hardware]
# Key generated: 0xABCD1234EFGH5678
```

Step 2: Verify Key Creation

```
# List public keys
gpg --list-keys

# Output:
# /home/user/.gnupg/pubring.gpg
# pub   4096R/EFGH5678 2024-01-15 [expires: 2026-01-15]
# uid   Alice Johnson <alice@example.com>
# sub   4096R/IJKL9012 2024-01-15 [expires: 2026-01-15]

# List private keys (secret keys)
gpg --list-secret-keys

# Output similar with "sec" instead of "pub"
```

Step 3: Export Public Key

```
# Export public key for sharing
gpg --armor --export alice@example.com > alice_public.gpg

# View exported key
cat alice_public.gpg

# Output:
# -----BEGIN PGP PUBLIC KEY BLOCK-----
# Version: GnuPG v2
# mQINBFxTMHkBEAC6m3...
# -----END PGP PUBLIC KEY BLOCK-----
```

Part C: Encrypt/Decrypt with GPG

Step 1: Create Test Message

```
# Create message to encrypt
cat > secret.txt << EOF
OPERATION RED ALERT
Location: Warehouse 7
Time: Midnight
Status: Critical
```

```
EOF
```

Step 2: Encrypt for Recipient

```
# Encrypt using Alice's public key
gpg --armor --recipient alice@example.com \
    --encrypt secret.txt

# Output: Creates secret.txt.asc (ASCII-armored encrypted file)

# Or use recipient's key ID:
gpg --armor --recipient 0xEFGH5678 --encrypt secret.txt

# View encrypted file
cat secret.txt.asc

# Output:
# -----BEGIN PGP MESSAGE-----
# Version: GnuPG v2
# hQIMA...
# -----END PGP MESSAGE-----
```

Step 3: Decrypt Message

```
# Decrypt (prompts for passphrase)
gpg --decrypt secret.txt.asc > secret_decrypted.txt

# Terminal prompts:
# You need a passphrase to unlock the secret key
# [Enter passphrase created during key generation]

# Verify decryption
cat secret_decrypted.txt

# Compare with original
diff secret.txt secret_decrypted.txt
```

Step 4: Demonstrate Key Dependency

```
# Attempt: Bob tries to decrypt message encrypted for Alice
# Using Bob's key, he will fail:

gpg --decrypt secret.txt.asc  # With Bob's key selected
# Error: No secret key available

# Proves: Only intended recipient can decrypt
```

Part D: Digital Signatures

Step 1: Create Document

```
# Create contract document
cat > contract.txt << EOF
```

```
EMPLOYMENT CONTRACT
Employee: John Smith
Position: Security Analyst
Salary: $95,000
Start Date: Feb 1, 2024
Approved by: Alice Johnson, Manager
Date: January 15, 2024
EOF
```

Step 2: Sign Document

```
# Create clearsigned document (readable + signature)
gpg --armor --clearsign contract.txt

# Output: contract.txt.asc (includes original + signature)

# View signed document
cat contract.txt.asc

# Output:
# -----BEGIN PGP SIGNED MESSAGE-----
# Hash: SHA256
#
# [Original document content]
#
# -----BEGIN PGP SIGNATURE-----
# Version: GnuPG v2
# iQIcBA...
# -----END PGP SIGNATURE-----
```

Step 3: Verify Signature

```
# Verify the signature
gpg --verify contract.txt.asc

# Output:
# gpg: Signature made Sun Jan 15 14:23:45 2024
# gpg:                using RSA key EFGH5678
# gpg: Good signature from "Alice Johnson <alice@example.com>"
# Primary key fingerprint: ABCD 1234 EFGH 5678 IJKL 9012
```

Step 4: Demonstrate Tamper Detection

```
# Modify the signed document
sed -i 's/95,000/105,000/' contract.txt.asc

# Attempt verification
gpg --verify contract.txt.asc

# Output:
# gpg: Signature made Sun Jan 15 14:23:45 2024
# gpg: BAD signature from "Alice Johnson <alice@example.com>"
# gpg: [GNUPG:] BAD_SIGNATURE
# gpg: [GNUPG:] KEY_CONSIDERED EFGH5678
```

Part E: Key Trust and Web of Trust

Step 1: Import Someone's Public Key

```
# Simulate receiving Bob's public key
gpg --import bob_public.gpg

# Verify import
gpg --list-keys bob@example.com
```

Step 2: Establish Trust Level

```
# Edit key to set trust level
gpg --edit-key bob@example.com

# At gpg> prompt:
# trust
# [Select trust level]
# 1 = I don't know or refuse to answer
# 2 = I do NOT trust
# 3 = I trust marginally
# 4 = I trust completely
# 5 = I trust ultimately

# Then: save
# (Establishes web of trust)
```

Deliverables

- Generated GPG key pair files
- Encrypted/decrypted message pairs
- Signed and verified documents
- Key management documentation
- Trust relationship diagrams

Lab Activity 6: Password Strength Analysis & Secure Password Generation

Difficulty: Beginner | Time: 45 minutes

Objectives

- Analyze password strength
- Understand secure password generation
- Test password against common attacks
- Implement password policies

Prerequisites

- Basic command line
- Optional: password testing tool (hashcat, hydra)

Step-by-Step Instructions

Part A: Analyze Password Strength

Step 1: Understand Password Entropy

Password entropy = $\log_2$(character_set^length)

```
# Entropy calculation examples:

# "password" (8 lowercase letters)
# Character set: 26 letters
# Entropy = log₂(26^8) = 37.6 bits
# Time to brute force (1 trillion guesses/sec): < 1 second

# "Tr0pic@l2024!" (13 characters, mixed case + symbols)
# Character set: 94 characters (upper, lower, digits, symbols)
# Entropy = log₂(94^13) = 86 bits
# Time to brute force: ~1 year

# "correct horse battery staple" (4 words)
# Character set: 170,000+ English words
# Entropy = log₂(170000^4) = 65 bits
# Time to brute force: ~1 month
```

Step 2: Test Sample Passwords

Create password_analysis.txt:

```
Password: password
Length: 8
Character types: lowercase
Entropy: 37.6 bits
Dictionary word: YES
Brute force risk: EXTREME
Rating: ❌ INSECURE

Password: MyP@ssw0rd2024
Length: 15
Character types: upper, lower, digits, symbols
Entropy: 98 bits
Dictionary word: NO
Brute force risk: VERY LOW
Rating: ☑ SECURE

Password: correct-horse-battery-staple
Length: 29
```

```
Character types: lower, hyphens
Entropy: 78 bits
Dictionary word: Passphrase (4 common words)
Brute force risk: LOW (if using word list)
Rating: ☑ ACCEPTABLE
```

Step 3: Tools for Password Strength Testing

```
# Install password strength testing tool
# Linux:
sudo apt-get install libpam-cracklib

# Use cracklib:
echo "TestPassword123" | cracklib-check

# Output:
# TestPassword123: OK

# Or for weak password:
echo "password" | cracklib-check
# Output:
# password: it is based on a dictionary word
```

Part B: Generate Secure Passwords

Step 1: Random Password Generation

```
# Method 1: Using /dev/urandom (Linux/macOS)
# Generate 16-character random password
head -c 32 /dev/urandom | base64

# Output: r5K9Lm2pQ8vX3nW7sY6jZ1gH4dF5vB9x

# Method 2: Using openssl
openssl rand -base64 16

# Output: aB9cD2eF5gH8iJ1kL4mN7oPqRsT0uVwX

# Method 3: Using Python
python3 -c "import secrets; import string; chars = string.ascii_letters +
string.digits + string.punctuation; pwd = ''.join(secrets.choice(chars) for i
in range(16)); print(pwd)"

# Output: Tr9@kL#mP2$xQvW%z
```

Step 2: Generate Passphrase (More Memorable)

```
# Using XKCD-style passphrase generator
# Method 1: Manual selection
# Words: (pick 4-5 random words from dictionary)
# Example: correct horse battery staple
# Add: numbers and symbols
# Result: correct-horse-2-battery-staple!
```

```
# Method 2: Automated (Linux)
# Install diceware:
sudo apt-get install diceware

# Generate 6-word passphrase
diceware -d - -n 6

# Output:
# abort-algebra-anchor-bing-cabin-dagger
# (Each word ~11 bits entropy, 6 words = 66 bits total)

# Add numbers/symbols:
# Result: abort-algebra-anchor-bing-cabin-dagger-2024!
```

Step 3: Implement Secure Password Policy

Create policy document:

```
SECURE PASSWORD POLICY

1. Minimum Length: 12 characters
2. Character Requirements:
   - At least 1 UPPERCASE letter
   - At least 1 lowercase letter
   - At least 1 digit (0-9)
   - At least 1 special character (!@#$%^&*)
   - NOT contain username or email

3. NOT Allowed:
   - Dictionary words (common words)
   - Keyboard patterns (qwerty, 123456)
   - Previous passwords (last 12)
   - Common passwords (top 1000 list)

4. Change Requirements:
   - Change every 90 days
   - Must be different from last 12 passwords
   - Cannot reuse old password for 12 months

5. Examples:
   ☑ SECURE: Tr0pic@lThund3r2024
   ☑ SECURE: C0rr3ct-H0rs3-B@tt3ry-St@ple
   ✗ INSECURE: password123
   ✗ INSECURE: MyName2024
   ✗ INSECURE: qwerty!@#
```

Part C: Test Against Common Attacks

Step 1: Dictionary Attack Simulation

```
# Create test password file
PASSWORD_HASH=$(echo -n "MyP@ssword123" | sha256sum)
echo "user1:$PASSWORD_HASH" > test_hashes.txt
```

```
# Create common words list
cat > common_words.txt << EOF
password
123456
password123
qwerty
admin
letmein
welcome
monkey
dragon
master
EOF

# Simple dictionary attack script (Bash):
#!/bin/bash
while IFS= read -r word; do
  HASH=$(echo -n "$word" | sha256sum | awk '{print $1}')
  if grep -q "$HASH" test_hashes.txt; then
    echo "Found: $word"
  fi
done < common_words.txt

# Result: Dictionary won't find MyP@ssword123 (too strong)
```

Step 2: Test with Hashcat

```
# Install hashcat (GPU-accelerated)
sudo apt-get install hashcat

# Generate test hash
TEST_HASH=$(echo -n "password" | sha256sum | awk '{print $1}')
echo "$TEST_HASH" > test.hash

# Dictionary attack
hashcat -m 1400 test.hash /usr/share/wordlists/rockyou.txt

# Output:
# [found] password
# [status] cracked: 1/1
```

Deliverables

- Password strength analysis document
- Password policy implementation
- Examples of secure vs. insecure passwords
- Generated secure passwords
- Test results from strength checking tools

Additional Resources & References

Tools Used in Labs

- OpenSSL: https://www.openssl.org/
- GPG/GnuPG: https://gnupg.org/
- John the Ripper: https://www.openwall.com/john/
- Hashcat: https://hashcat.net/
- Wireshark: https://www.wireshark.org/

Standards & Best Practices

- NIST Cybersecurity Framework: https://www.nist.gov/cyberframework
- OWASP Top 10: https://owasp.org/www-project-top-ten/
- CIS Benchmarks: https://www.cisecurity.org/
- SANS Security Resources: https://www.sans.org/
- RFC 5652 (S/MIME): https://tools.ietf.org/html/rfc5652

Further Learning

- "Cryptography Engineering" by Ferguson, Schneier, Kohno
- "Web Application Security" by Stuttard & Pinto
- SANS SEC401: Security Essentials
- Coursera Cryptography Courses
- Cybrary Ethical Hacking Path

The End!